POLITECNICO DI BARI

DEPARTMENT OF ELECTRICAL AND INFORMATION ENGINEERING
Master Degree in Automation Engineering

Dissertation in Mobile Robotics

"Autonomous Navigation Simulator with Lane Keeping and Obstacle Avoidance"

Supervisors
Prof. Luca De Cicco
Nunzio Barone

Candidates
Stefano Di Lena
Marco Cappelluti Pappagallo
Nicolas Nicoletti

Academic Year 2025 - 2026

Contents

1 Introduction **1**
1.1 Problem Formulation 1
1.1.1 Approach 1

2 Modeling Assumptions **2**
2.1 Kinematic Model 2
2.1.1 Controllability Analysis 3
2.2 Sensor Model 3

3 Algorithmic Choices **4**
3.1 Software Architecture 4
3.2 Perception 4
3.3 Motion Control and I/O Linearization 5

4 Experimental Validation **6**
4.1 Simulation Environment Setup 6
4.2 Tracking Performance and Controller Stability 6
4.3 Finite State Machine Responsiveness 7

5 Critical Discussion of Results and Conclusions **8**
5.1 System Efficacy 8
5.2 Limitations of the I/O Linearization 8
5.3 Limitations of the Reactive Paradigm 8
5.4 Real-World Transferability and State Estimation 9
5.5 Conclusions 9

Chapter 1

Introduction

1.1 Problem Formulation

The core objective of this project is to develop and implement an autonomous navigation system for a non-holonomic Differential Drive Robot (DDR) operating within a closed-loop circuit. The mobile robot should be able to navigate the track safely, avoiding static obstacles (parked vehicles and narrow passages) placed along the path.

1.1.1 Approach

We used a reactive paradigm rather than relying on a pre-computed global map. The robot makes instantaneous driving decisions based exclusively on the data of the LiDAR sensor onboard. The problem is formulated as a local trajectory tracking structured around a Finite State Machine coupled with a PID controller.

Given the reactive focus of this approach and the structured nature of the environment, global path planning algorithms and Simultaneous Localization and Mapping (SLAM) techniques were intentionally excluded. Relying solely on local spatial constraints allows for a significant reduction in computational overhead, ensuring high-frequency responsiveness to immediate hurdles without the need for absolute global positioning.

Chapter 2

Modeling Assumptions

2.1 Kinematic Model

The locomotion of the platform is based on a Differential Drive Robot (DDR) architecture. The mobility of the chassis is strictly governed by the physical constraints imposed by its wheels, specifically the pure rolling and no-sliding conditions. Mathematically, the no-sliding constraints for all wheels can be aggregated into a single Pfaffian constraint equation, expressed as $C_1(\beta_s)R(\theta)\dot{q}_I = 0$.

For a DDR, which possesses $N = 2$ standard fixed wheels and zero steerable wheels ($\delta_s = 0$), this constraint ensures that the instantaneous velocity along the robot's lateral axis (Y_R) is strictly zero ($\dot{y} = 0$). The forward kinematic model defines the relationship between the wheels' angular velocities and the robot's pose in its local reference frame. Assuming both wheels share the same radius r and are located at a distance l from the central axis P, the kinematic equations are derived as follows:

$$\dot{x} = \frac{1}{2}r\dot{\varphi}_1 + \frac{1}{2}r\dot{\varphi}_2$$

$$\dot{\theta} = \frac{r\dot{\varphi}_1}{2l} - \frac{r\dot{\varphi}_2}{2l}$$

Where $\dot{\varphi}_1$ and $\dot{\varphi}_2$ represent the angular speeds of the right and left wheel, respectively.

Because the degree of mobility is $\delta_m = 2$, the DDR is mathematically equivalent to the kinematic model of an Unicycle. By defining the forward linear velocity as $v = \dot{x}$ and the angular velocity as $\omega = \dot{\theta}$, the system can be fully controlled using these two decoupled virtual inputs. This mathematical mapping justifies the unicycle-based control logic adopted in our algorithm.

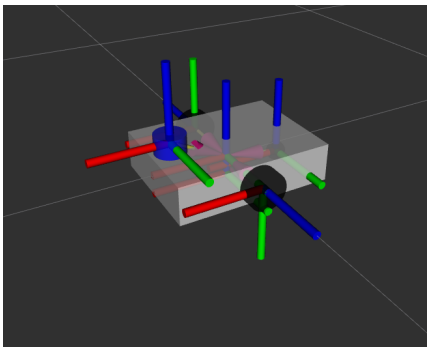

Figure 2.1: Differential Drive Robot in RViz.

2.1.1 Controllability Analysis

A critical theoretical aspect of the unicycle model involves its controllability properties, which depend heavily on the allowed control input domains. When the robot is permitted to command both forward and backward linear velocities, the non-holonomic system is classified as Small-Time Locally Controllable (STLC). An STLC system guarantees extreme agility, as it can maneuver to reach any nearby configuration in an arbitrarily short amount of time by exploiting Lie bracket motions.

If the robot were mechanically or algorithmically restricted to forward-only motion, the system would degrade to being STLA (Small-Time Locally Accessible). While an STLA system can still eventually reach the desired states, it loses the theoretical guarantee of doing so within small time frames and requires significantly more complex maneuvering. Retaining the STLC property by allowing zero or reverse linear velocities provides the extreme maneuverability required to effectively and safely negotiate tight spaces, such as the narrow passages encountered in the final sections of the circuit.

2.2 Sensor Model

To perceive the surrounding environment, the robot is equipped with a 2D LiDAR (Light Detection and Ranging), an active exteroceptive sensor. The LiDAR operates on the Time-of-Flight (ToF) principle: it emits highly focused infrared laser pulses and detects the reflections bouncing back from nearby surfaces. By measuring the precise round-trip time (Δt) of each pulse, the sensor computes the distance d to the obstacle using the constant speed of light c, according to the formula:

$$d = \frac{c \cdot \Delta t}{2}$$

Through an internal rotating mechanism, the sensor sweeps the laser beam across a planar field, generating a dense 2D point cloud natively represented in polar coordinates (distance and angle). For the scope of this project, we assume a forward-facing 180° field of view.

To translate this raw, high-dimensional data array into actionable variables for reactive navigation, a specific geometric extraction logic is applied. The 180° scan is partitioned into three distinct angular Regions of Interest: a front sector, a left sector, and a right sector. Rather than processing the entire point cloud for complex obstacle reconstruction, the algorithm filters the arrays to extract the minimum distance to the nearest physical boundary within each specific regions.

Chapter 3

Algorithmic Choices

3.1 Software Architecture

The software architecture of the autonomous system was developed using ROS 2 (Robot Operating System), ensuring a highly modular and scalable framework. The system is conceptually divided into two processing units:

- `perception_node`;
- `control_node`.

Communication between these nodes is governed by the Publish/Subscribe paradigm utilizing ROS 2 Topics.

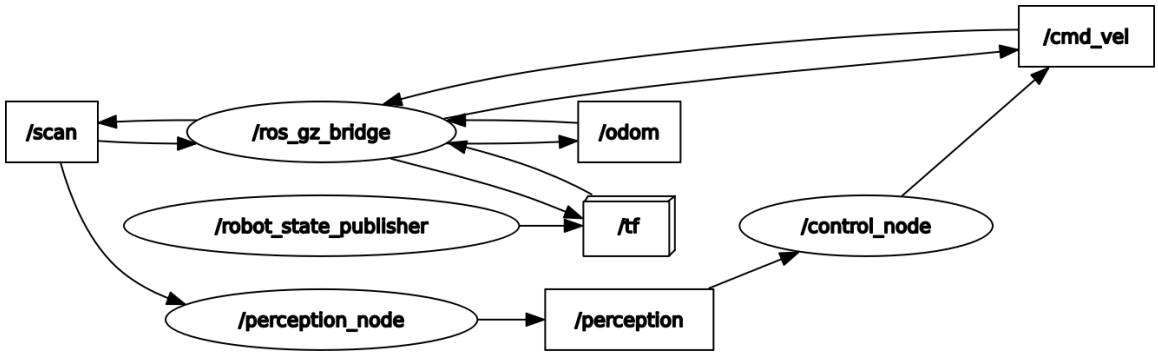

Figure 3.1: Computational grph illustrating the asynchronous data flow between the sensory pipeline, the custom control node and the Gazebo bridge.

3.2 Perception

The `perception_node` subscribe to the raw sensor data streams (sensor_msgs/LaserScan) and abstract the complex 2D point cloud into the actionable metrics required by the custom PerceptionData message. Within this node, the raw 180° laser scan array is partitioned into the three predefined Regions of Interest: front, left, and right. The pipeline algorithm iterates through the ranges array, filtering out noise and invalid readings (such as infinite values or reflections outside the maximum range). These three distance values are packed into the message PerceptionData and continuously published to the /perception topic at the sensor's native update rate, providing a constantly refreshed geometric abstraction of the environment.

3.3 Motion Control and I/O Linearization

To apply a standard linear PID controller, an Input-Output (I/O) Linearization technique was implemented. Attempting to linearize the system directly at the center of the wheel axle leads to a mathematical singularity, as the control matrix becomes non-invertible. To solve this, the control point is shifted forward along the robot's longitudinal axis to a new virtual point located at a distance b m from the center.

Let the Cartesian coordinates of the center be (x, y) and the robot's orientation be θ. The coordinates of the new control point are defined as:

$$x_B = x + b\cos\theta$$

$$y_B = y + b\sin\theta$$

By differentiating these equations with respect to time, we obtain the relationship between the velocities of point B and the physical control inputs of the unicycle model (linear velocity v and angular velocity ω):

$$\begin{bmatrix} \dot{x}_B \\ \dot{y}_B \end{bmatrix} = \begin{bmatrix} \cos\theta & -b\sin\theta \\ \sin\theta & b\cos\theta \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix}$$

The transformation matrix is known as the decoupling matrix $T(\theta)$. The determinant of this matrix is:

$$\det(T(\theta)) = b(\cos^2\theta + \sin^2\theta) = b$$

This explicitly demonstrates the necessity of the offset: by choosing $b = 0.15$ m, we guarantee that the determinant is strictly non-zero ($\det \neq 0$), avoiding the mathematical singularity and ensuring the matrix is always invertible.By defining the virtual control inputs computed by the FSM and PID controller as u_x (forward velocity command) and u_y (lateral corrective command), the inverse relationship yields the exact control law applied to the robot:

$$\begin{bmatrix} v \\ \omega \end{bmatrix} = \begin{bmatrix} \cos\theta & \sin\theta \\ -\frac{\sin\theta}{b} & \frac{\cos\theta}{b} \end{bmatrix} \begin{bmatrix} u_x \\ u_y \end{bmatrix}$$

By evaluating this control law strictly within the robot's local reference frame (where the relative orientation is always $\theta = 0$), the trigonometric components simplify, yielding the final decoupled actuation formulas utilized in the node's source code:

$$v = u_x$$

$$\omega = \frac{u_y}{b}$$

These resulting unicycle velocities, v and ω, are subsequently packed into a standard geometry_msgs/Twist message and published to the /cmd_vel topic, directly driving the robotic platform within the simulated environment.

Chapter 4

Experimental Validation

4.1 Simulation Environment Setup

The experimental validation of the proposed autonomous navigation system was conducted using the Gazebo physics simulator. The testing environments are two custom .sdf world file, designed to replicate distinct challenging scenarios (one is a continous closed-loop circuit intended for uninterrupted navigation tests; the other is a shorter finite track with a higher density of obstacles and a defined endpoint). To rigorously test the system's reactive capabilities, both tracks are intentionally populated with parked vehicles acting as static obstacles and a narrow passage that heavily restricts the navigable lateral space.

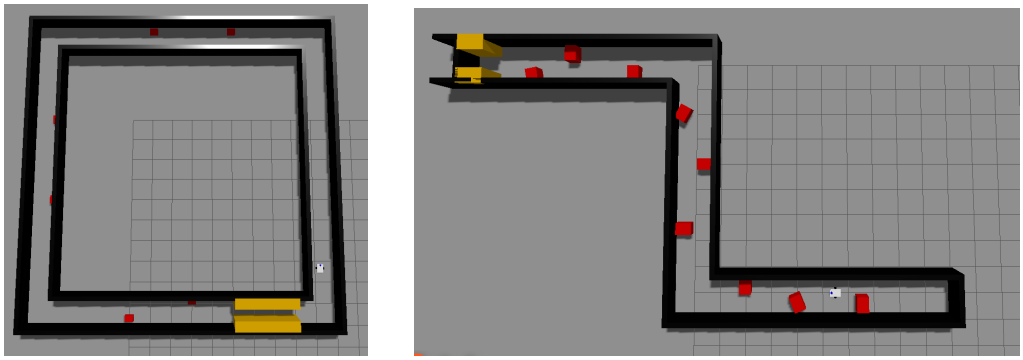

Figure 4.1: Top-down view of the two Gazebo environments used for the simulation.

4.2 Tracking Performance and Controller Stability

To validate the PID controller's stability, we tracked the lateral error and the angular velocity command ω over time.

As shown in the graph, during normal lane-following, the error remains at zero. When the robot performs an overtake the lateral error suddenly increase. The PID controller reacts instantly to correct the trajectory (the angular velocity quickly ramps up, hitting our limit without exceeding it. The error and the steering command smoothly return to zero in few seconds; this proves that the system is stable and do not suffer from continous zig-zag oscillations.

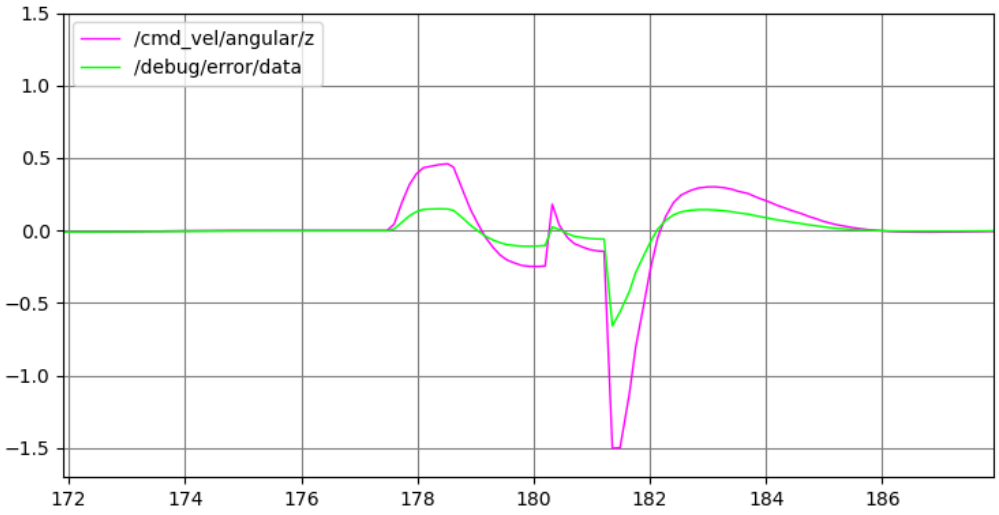

Figure 4.2: Plot of the lateral error (green line) and the corresponding angular velocity command generated by the control node (magenta line).

4.3 Finite State Machine Responsiveness

To verify the Finite State Machine (FSM), we plotted the frontal distance from the LiDAR against the robot's forward speed.

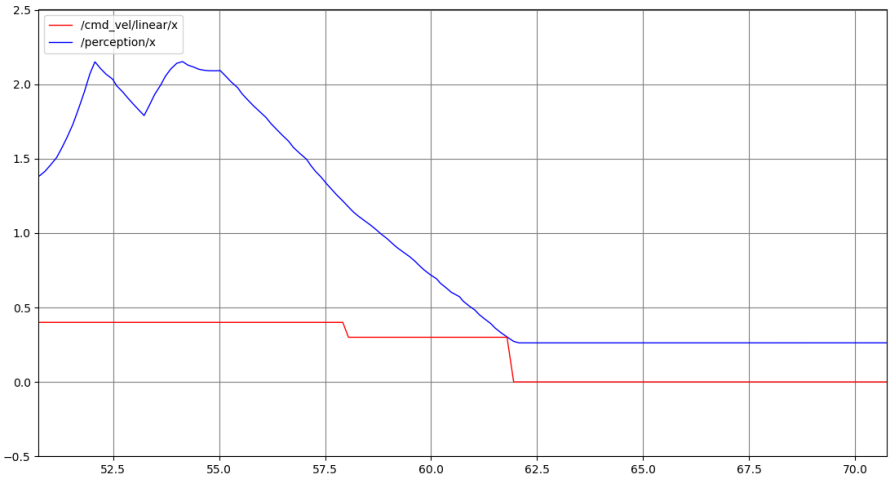

Figure 4.3: Correlation between the perceived frontal distance (blue line) and the linear velocity (red line).

The graph clearly demonstrates how the FSM reacts to an approaching obstacle in real-time. Around $t \approx 58$ s, when the frontal distance drops below the 1.2 m safety threshold, the robot automatically slows down from its 0.4 m/s cruise speed to 0.3 m/s to prepare for the narrow passage. Later, at $t \approx 62$ s, the distance hits the critical 0.3 m threshold. The graph shows the forward speed dropping instantly to 0.0 m/s. This confirms that the emergency brake triggers perfectly, guaranteeing the robot stops before a mechanical collision occurs.

Chapter 5

Critical Discussion of Results and Conclusions

5.1 System Efficacy

The proposed autonomous navigation system successfully fulfilled the project's primary objectives. The hierarchical architecture, combining a reactive Finite State Machine with an I/O Linearization PID controller, proved to be highly effective for navigating a structured closed-loop circuit. The robot consistently maintained its lane, executed overtaking maneuvers, and respected critical. However, a critical analysis of the underlying theoretical assumptions reveals specific structural limitations that define the boundaries of this approach.

5.2 Limitations of the I/O Linearization

While the Input-Output Linearization solves the non-holonomic control problem by avoiding singularity, it introduces a trade-off regarding the robot's internal dynamics. By shifting the control point from the center P to a virtual point B located at $b = 0.15$ m, the controller perfectly regulates the Cartesian position (x_B, y_B). However, this mathematical abstraction means the exact orientation θ of the robot becomes an unobservable state from the controller's perspective (often referred to as zero dynamics). Consequently, while point B accurately tracks the desired path, the robot's main chassis might exhibit slight, uncontrolled angular sweeping motions, especially during sharp corrective maneuvers.

5.3 Limitations of the Reactive Paradigm

The reactive navigation paradigm implemented is highly computationally efficient but fundamentally short-sighted. Because the driving logic relies exclusively on instantaneous local data without historical memory or a global map, the system is highly vulnerable to "local minima." For instance, if the robot were to enter a deep "U-shaped" obstacle, the current algorithm would likely oscillate indefinitely between avoiding the left, right, and front walls, unable to logically reverse out of the trap. In such complex, unstructured scenarios, the reactive approach must be entirely superseded, or heavily augmented, by global path planning algorithms such as Rapidly-exploring Random Trees or Probabilistic Roadmaps.

5.4 Real-World Transferability and State Estimation

The current Gazebo simulation assumes idealized sensor data and perfect odometry. In a real-world deployment, LiDAR readings are subjected to Gaussian noise, and wheel encoders suffer from mechanical slippage, leading to dead-reckoning drift over time.The current architecture lacks a state estimation layer to filter these inaccuracies. To deploy this system on physical hardware, it would be strictly necessary to implement an Extended Kalman Filter (EKF). An EKF would fuse the raw LiDAR inputs with wheel odometry and an Inertial Measurement Unit (IMU), providing a filtered, robust estimation of the robot's true state. Furthermore, the algorithm currently assumes a purely kinematic model, disregarding the robot's mass, inertia, and motor torque limits. Accounting for these dynamic factors would be required to prevent the PID controller from requesting physically unachievable accelerations.

5.5 Conclusions

In conclusion, the developed reactive control architecture represents a lightweight solution for predictable and structured environments that require rapid obstacle avoidance. The system ensured robust lane keeping, dynamic obstacle avoidance, and strict compliance with safety constraints (Emergency Stop) in real time, while maintaining high computational efficiency. The developed software architecture is a basic module, ready to be extended with Global Path Planning algorithms (such as PRM or RRT) and Sensor Fusion techniques (such as Extended Kalman Filter), highlighting the transition from purely kinematic and local reactivity to dynamic and globally aware autonomous navigation.

Bibliography

[1] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*. MIT Press, second ed., 2011.

[2] Open Source Robotics Foundation, "ROS 2 Documentation: Jazzy Jalisco." `https://docs.ros.org/en/jazzy/`, 2024.